

실전 기계학습 OpenMP 프로젝트 보고서

학번/이름	2021105654/최원준
-------	----------------

1. 프로젝트 개요

본 프로젝트는 C++로 SRCNN(Super-Resolution Convolutional Neural Network)을 구현하고, OpenMP를 활용하여 CNN의 주요 연산(Convolution, ReLU 등)을 병렬화하는 것을 목표로 한다. Tensor3D, Layer, Model 클래스를 직접 설계·구현하였으며, forward/backward pass 및 SGD 기반 학습까지 포함한다.

2. 이론적 배경

SRCNN (Dong et al., 2016)은 저해상도 이미지를 고해상도로 복원하는 CNN 모델이다.

네트워크는 3단계 구조로, 각각 하나의 Conv 레이어에 대응된다.

1. **Patch extraction & representation** : LR 이미지에서 패치를 추출하고 feature map으로 표현
2. **Non-linear mapping** : LR feature $\rightarrow$ HR feature로 비선형 매핑
3. **Reconstruction** : HR feature를 조합해 최종 HR 이미지 복원

전통적인 sparse coding 기반 SR 파이프라인이 이 3-layer CNN과 수학적으로 동치임을 이론적으로 보임

Sparse Coding SR과 3-layer CNN의 동치 관계

Sparse Coding 기반 SR은 다음 3단계 파이프라인으로 이루어진다

1단계 — LR 패치를 LR 딕셔너리에 투영 (= Conv Layer 1)

입력 이미지에서 $f_1 \times f_1$ 크기 패치를 추출하고, n_1 개의 LR 딕셔너리 원소들과 내적(projection)한다.

이는 수학적으로 n_1 개의 $f_1 \times f_1$ 필터로 이미지를 convolution하는 것과 동일함.

mean subtraction 같은 전처리도 선형 연산이므로 convolution에 흡수된다.

- $f_1 \times f_1$ 크기 패치는 커널 전체를 구성하는 컨볼루션 슬라이스들에 대응
- LR 딕셔너리는 n_1 개의 $f_1 \times f_1$ 크기 패치들, Conv1 커널에 대응

2단계 — Sparse Coding Solver로 계수 변환 (= Conv Layer 2의 비선형 매핑)

LR 계수 n_1 개 $\rightarrow$ HR 계수 n_2 개로 변환한다.

Sparse coding solver(e.g., Feature-Sign)는 1×1 spatial support를 가진 비선형 매핑 연산자의 특수한 경우로 볼 수 있습니다.

다만 solver는 반복적(iterative) 알고리즘이지만, SRCNN의 Conv Layer 2는 feed-forward라 훨씬 효율적입니다.

- LR 계수 n_1 개 : Conv1 출력 feature map 64장 $\rightarrow$ Conv2의 입력에 대응
- HR 계수 n_2 개 : Conv2 출력 feature map 32장 $\rightarrow$ Conv3의 입력에 대응

3단계 — HR 디서너리로 복원 후 평균 (= Conv Layer 3)

n_2 개의 계수를 HR 디서너리에 투영해 HR 패치를 만들고 겹치는 부분을 평균낸다.

이는 $f_3 \times f_3$ 필터로 n_2 개의 feature map을 convolution하는 것과 동일합니다.

- HR 디서너리는 Conv3의 커널($f_3 \times f_3$ 크기, n_2 개 채널)에 대응됨

Sparse Coding SR과 SRCNN 비교

	Sparse Coding SR	SRCNN
최적화 대상	LR 디서너리, HR 디서너리만 개별 최적화	3단계 전체를 통합 end-to-end 최적화
속도	Solver가 iterative $\rightarrow$ 느림	완전 feed-forward $\rightarrow$ 빠름

즉, 구조 자체는 같지만 Sparse Coding은 각 단계를 따로따로 최적화한 반면,

SRCNN은 세 레이어를 한꺼번에 같이 학습합니다. 이 통합 최적화가 성능 향상의 핵심 이유이다

OpenMP

OpenMP는 공유 메모리 환경에서의 병렬 프로그래밍을 위한 API이다

`#pragma omp parallel for` 지시문을 통해 반복문을 여러 스레드로 분할 처리한다.

`collapse(n)`을 사용하면 중첩 반복문을 하나의 병렬 루프로 처리할 수 있어 더 많은 병렬성을 확보할 수 있다.

3. 구현 내용

클래스 구조

Tensor3D

- $nH \times nW \times nC$ 크기의 3D 텐서를 flat 1D 배열(double*)로 관리.
- 인덱스 = $h \times nW \times nC + w \times nC + c$
- 연속 메모리 구조로 캐시 효율을 높임

Layer(추상 클래스): forward/backward/update_weights 인터페이스 정의. virtual 함수로 다형성 구현

- Layer_ReLU
 - Layer_ReLU::forward()
 - 양수는 그대로 통과
 - 음수를 0으로 치환
 - cached_output으로 backward 마스크 관리
 - Layer_ReLU::backward()
 - cached_output에 저장해둔 forward 출력값을 마스크로 활용하여,

cached_output > 0인 위치는 grad_output을 그대로 전달하고, 그렇지 않은 위치는 0을 반환
 - Layer_ReLU::update_weights()
 - 가중치가 없는 layer(ReLU)는 기본 동작 없음
- Layer_Conv
 - weight_tensor, bias_tensor, grad_weight_tensor, grad_bias_tensor를 가짐
 - weight_tensor 크기 : $fK \times fK \times fC_{in} \times fC_{out}$, bias_tensor 크기 : fC_{out} , 모두 flat 배열로 관리
 - weight_tensor 인덱스 = $((ph \times fK + pw) \times fC_{in} + ci) \times fC_{out} + co$
 - grad도 같은 방식
 - MEAN_INIT/LOAD_INIT 두 가지 초기화 모드 지원
 - Layer_Conv::init()
 - init_type (MEAN_INIT 또는 LOAD_INIT)에 따라 가중치를 다른 방식으로 초기
 - MEAN_INIT : 모든 가중치를 $1/(fKfKfC_{in})$ 으로, bias는 0으로 초기화
 - LOAD_INIT : 파일에서 값을 읽어 가중치에 저장
 - Layer_Conv::forward()
 - 각 출력 위치 (h, w, co)에 대해 $fK \times fK \times fC_{in}$ 크기의 커널과 컨볼루션 연산을 한 뒤, bias를 더한다

- 패딩을 추가하지 않고, 커널이 입력 범위를 벗어나지 않는 유효한 위치(offset ~ nH-offset-1)만 계산한다. 출력 텐서는 입력과 동일한 크기로 할당되며, 계산되지 않은 가장자리는 0으로 유지된다.
 - backward에서 dL/dW 계산에 쓰기 위해 입력을 `cached_input`에 복사하여 저장한다.
- `Layer_Conv::backward()`
 - forward 수식으로부터 세 가지 gradient를 계산한다
- `Layer_Conv::update_weights()`
 - SGD(Stochastic Gradient Descent) 방식으로 가중치를 업데이트한다

Model

- layers/tensors 벡터로 전체 파이프라인 통합 관리
- `Model::test()`
 - 입력 이미지를 읽어 `tensors[0]`에 저장한 뒤,
 - 순서대로 각 레이어의 forward를 수행하여 결과를 tensors에 축적한다. (구현 부분)
 - 최종 출력 tensor를 이미지로 변환하여 저장한다.
 - 학습 없이 순수 추론만 수행
- `Model::train()`
 - 입력 이미지와 타겟 이미지를 각각 읽어 tensor로 변환한 뒤, epochs 횟수만큼 `train_step()`을 반복 호출한다.
 - 타겟 tensor는 루프 밖에서 한 번만 생성하여 재사용한다.
- `Model::train_step()`

1 epoch의 학습을 수행하며 `omp_get_wtime()`으로 각 단계 시간을 측정한다.

- **(a) forward pass:** `tensors[i] → layers[i]->forward() → tensors[i+1]` 순서로 전파
- **(b) loss** 계산: MSELoss로 pred와 target의 오차 계산. $MSE\ Loss: L = (1/N) * \sum((pred - target)^2)$
- **(c) backward pass:** loss gradient로부터 역방향으로 각 레이어의 backward 호출. grad를 뒤에서 앞으로 전달하며 중간 grad는 즉시 해제
- **(d) weight** 업데이트: 모든 레이어의 `update_weights(lr)` 호출
- **(e) 중간 tensor** 정리: `tensors[0]`(입력)만 유지하고 나머지 해제

SRCNN 구조

(Layer information)

1-th layer: Conv1 9x9x1x64

2-th layer: Relu1 1x1x64x64

3-th layer: Conv2 5x5x64x32

4-th layer: Relu2 1x1x32x32

5-th layer: Conv3 5x5x32x1

OpenMP 병렬화 적용 위치

- `Layer_ReLU::forward/backward()`: `#pragma omp parallel for collapse(3)` — h, w, c 루프 병렬화
- `Layer_Conv::forward()`: `#pragma omp parallel for collapse(2)` — h, w 루프 병렬화
- `Layer_Conv::backward()`: `#pragma omp parallel for collapse(2)` — h, w 루프 병렬화. $dL/dX, dL/dW$ 누적 시 `#pragma omp atomic`으로 race condition 방지
- `Layer_Conv::update_weights()`: `#pragma omp parallel for` — weight 업데이트 병렬화

4. 결과 및 분석

구분	1 Thread	2 Thread	4 Thread	Speed up (2 Thread)	Speed up (4 Thread)
Forward pass	59.6586s	30.8255s	19.2206s	1.9353x	3.1038x
Backward pass	116.643s	66.5831s	49.0208s	1.7518x	2.3794x
Weight update	0.000116825s	0.000202894s	0.000106812s	0.5757x	1.093x
Total(1 epoch)	176.312s	97.4199s	68.2532s	1.80981x	2.5832x

스레드 수가 늘어날수록 처리 속도가 향상되었으나, 이상적인 선형 Speedup(2배, 4배)에는 미치지 못하였다. 이는 `#pragma omp atomic`으로 인한 동기화 오버헤드, 특히 backward pass에서 $dL/dX, dL/dW$ 누적 연산 시 발생하는 경쟁 조건(race condition) 처리 비용이 주요 원인으로 분석된다. Forward는 스레드 수 증가에 따라 비교적 선형에 가까운 성능 향상을 보였으나, Backward의 atomic 연산이 병렬화 효율을 제한하였다.

5. 코드 첨부

첨부한 코드 파일 참고

6. 참고문헌 / 자료

- [1] Dong et al., "Image Super-Resolution Using Deep Convolutional Networks," TPAMI, 2016.
- [2]. OpenMP 공식 문서: <https://www.openmp.org/specifications/>